UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BACHELOR'S DEGREE

Bachelor Thesis

by

Your Name

Your Student ID

Information and Communication Technology

Title:

Design and Implementation of Anhoc

A Web-Based Mathematics Learning Platform for Structured Learning, Adaptive Practice, and Progress Tracking

Supervisor:	Your Supervisor
Project basis:	Anhoc web application repository
Implementation scope:	`frontend/`, `backend/`, and `chatbot/`

Ho Chi Minh City. June 2026

Declaration

I hereby declare that this thesis titled *Design and Implementation of Anhoc* is prepared from the implemented Anhoc repository and the accompanying project documentation. All architectural descriptions, feature summaries, and implementation details presented in this report are derived from the inspected codebase, and external concepts or technologies are cited where appropriate.

To the best of my knowledge, this report does not intentionally reproduce uncited work as an original contribution. Any frameworks, libraries, documents, or external references used in the preparation of this thesis are acknowledged in the bibliography.

Ho Chi Minh City, June 2026

Your Name

Acknowledgements

This thesis was prepared from the current Anhoc web application repository. The document reflects the work embodied in the frontend, backend, database, and chatbot modules, as well as the supporting technical design notes maintained with the project.

I would like to acknowledge the contributors who shaped the platform's architecture, educational workflows, and deployment configuration, together with the maintainers of the open-source tools that made the system possible, including Next.js, React, Express, Prisma, PostgreSQL, and the supporting Python ecosystem used by the tutoring service.

(Your Name)
Ho Chi Minh City, June 2026

Contents

List of Acronyms

API	Application Programming Interface
BYOK	Bring Your Own Key
CORS	Cross-Origin Resource Sharing
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
UI	User Interface
XP	Experience Points

List of Figures

List of Tables

Abstract

Anhoc is a web-based mathematics learning platform designed for structured school-level study. The implemented system combines bilingual lesson content, dynamically generated practice exercises, grade-level tests, progression tracking, administrative management, notifications, multiplayer and single-player math games, and an integrated tutoring chatbot. This thesis documents the design and implementation of Anhoc based on the current repository contents.

The software follows a service-oriented web architecture. A Next.js frontend provides the student and administrative interfaces. An Express backend exposes REST endpoints for authentication, lessons, tests, achievements, notifications, games, supervisor workflows, and subscription management. Prisma is used as the persistence layer, PostgreSQL stores the core application state, and a companion Python chatbot service extends the platform with conversational tutoring capabilities.

A notable implementation decision is the use of persisted question snapshots. Reusable templates define how questions are generated, while snapshots preserve the exact generated variables, accepted answers, and scoring outcomes for each attempt. This design supports reproducible grading, debugging, review, and reporting. The project also adopts dynamic role-based access control with subject-level restrictions, progression services for mastery and XP, and operational support for learn units and subscription-oriented platform management.

This thesis presents the problem context, requirements, system architecture, data model, implementation details, and an evaluation of the current solution. It also discusses limitations and practical improvements required to mature the platform further as an educational software system.

Keywords: mathematics learning platform, question generation, role-based access control, progress tracking, tutoring chatbot, web application architecture

Chapter 1

Introduction

1.1 Background

Digital learning systems are increasingly expected to do more than present static content. In mathematics education, students need repeated practice, immediate feedback, structured progression, and content that can be adapted to multiple grades and topics. A conventional content website can display lessons, but it does not by itself provide reusable question generation, mastery tracking, or administrative control over subject-specific learning material.

The Anhoc project addresses this gap by combining theory lessons, generated practice, formal test sessions, and gamified progression in a single web application. The project targets school-level mathematics learning and is organized around subjects, grades, and lessons. The implemented system allows a student to read theory content, start practice from lesson templates, submit answers, complete tests, and monitor progress through XP, levels, and achievements. At the same time, an administrator can manage lessons, question templates, user accounts, roles, and access permissions.

1.2 Problem Statement

The main engineering problem solved by Anhoc is the design of a learning platform that satisfies four requirements at the same time:

- it must support reusable content across grades and subjects;
- it must generate exercises dynamically rather than storing every question instance manually;

- it must keep each attempt stable enough to be scored, reviewed, and audited later;
- it must provide operational control for administrators without hard-coding every permission into the user interface.

Many simple practice applications solve only one part of this problem. They either store fixed question banks, do not track long-term student progression, or rely on coarse user roles that become difficult to extend. Anhoc introduces a more structured approach by combining generated questions with persisted snapshots, dynamic role-based access control, and a progression subsystem.

1.3 Objectives

This thesis has the following objectives:

- document the implemented Anhoc system in a thesis-style format;
- explain the architecture and data model used by the project;
- describe how the platform generates and evaluates mathematical exercises;
- analyze how authentication, authorization, and administration are handled;
- evaluate the strengths, limitations, and next steps of the current implementation.

1.4 Scope

The scope of this thesis is limited to the code and documentation available in the repository. It focuses on the currently implemented frontend, backend, database schema, and operational flows. The thesis does not claim to measure learning outcomes through classroom experimentation, because no such experimental dataset is included in the repository. Evaluation is therefore based on software capabilities, system structure, and scenario-level validation.

1.5 Contributions of the Project

The implemented project contributes several concrete software ideas:

- a bilingual lesson structure for educational content in English and Vietnamese;

- a question-template model that supports variables, derived expressions, constraints, and multiple accepted formulas;
- persisted question snapshots that separate reusable templates from concrete attempt data;
- a progression model combining mastery, XP logs, levels, and achievements;
- a dynamic access-control model that separates actions, resources, and subject restrictions.

1.6 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 presents the requirements and analysis of the project domain. Chapter 3 describes the system design, architecture, security model, and data model. Chapter 4 explains implementation details across the frontend, backend, and database layers. Chapter 5 evaluates the current solution through representative scenarios, strengths, and limitations. Chapter 6 concludes the thesis and outlines future work.

Chapter 2

Requirements Analysis

2.1 Stakeholders and User Roles

The primary stakeholders in Anhoc are students, administrators, and maintainers of the platform.

- Students consume learning content, complete exercises, take tests, and monitor progress.
- Administrators maintain educational content, templates, users, roles, and reports.
- Maintainers operate the deployment, database, and system configuration.

The codebase also distinguishes between global roles and subject-specific permissions. This is important because a role may have permission to manage some resources without necessarily having access to all subjects.

2.2 Functional Requirements

Table 2.1 summarizes the main functional requirements derived from the implementation.

Table 2.1: Core functional requirements

Area	Requirement
Authentication	The system shall allow user registration, login, profile retrieval, password change, and activity retrieval.
Lessons	The system shall support subject and grade organization, lesson listing, lesson detail retrieval, and lesson authoring by authorized users.
Practice	The system shall generate lesson-based practice attempts from reusable templates and store concrete snapshot records for each generated question.
Testing	The system shall create grade-level test attempts, support answer submission, and compute final scores on completion.
Templates	The system shall support creation, update, deletion, preview, reporting, and review of question templates.
Progression	The system shall track study time, mastery status, XP, levels, and achievements.
Administration	The system shall provide user, role, permission, subject-access, and dashboard management endpoints.

2.3 Non-Functional Requirements

The repository implies the following non-functional requirements:

- **Modularity:** frontend, backend, persistence, and progression logic should remain separable.
- **Consistency:** generated questions must remain stable after creation so that grading and reporting are reproducible.
- **Security:** protected endpoints require JWT-based authentication and permission checks.
- **Extensibility:** roles, actions, resources, and subject restrictions should be data-driven rather than hard-coded.
- **Deployability:** the architecture should run in a common cloud arrangement using Vercel, Render, and Neon.

2.4 Learning Content Requirements

Educational content in Anhoc is organized around a hierarchy of subjects, grades, and lessons. Lessons contain bilingual titles and markdown-based content bodies.

This leads to several domain requirements:

- one lesson belongs to one grade and one subject;
- lesson content should be readable in more than one language;
- practice content should be attached to lessons through question templates rather than embedded into the lesson body itself;
- progress should be measurable at the lesson level.

2.5 Assessment Requirements

Assessment in the project has two modes:

- lesson-based practice for targeted repetition;
- grade-level tests for broader assessment across available lesson templates.

The assessment subsystem must support question generation, answer submission, correctness evaluation, score calculation, and post-completion progression updates. The implementation also needs to distinguish between theoretical questions and computed math questions because the grading strategy differs between those two cases.

2.6 Administrative Requirements

Administrative functionality in Anhoc goes beyond simple content editing. The platform must manage:

- users and their assigned roles;
- permissions by action and resource;
- subject-level restrictions per role;
- dashboard statistics and user insights;
- reports submitted for problematic generated questions.

This requirement set justifies the presence of a dynamic RBAC model instead of a fixed boolean admin flag.

2.7 Operational Assumptions

The codebase assumes a web-first deployment model:

- the frontend is delivered as a Next.js application;
- the backend exposes REST endpoints under `/api/v1`;
- PostgreSQL is the source of truth for transactional and analytical learning data;
- environment variables define API URLs, CORS origins, JWT secret material, and achievement seeding behavior.

These assumptions shape the rest of the architecture described in the next chapter.

Chapter 3

System Design

3.1 Architectural Overview

Anhoc uses a three-tier web architecture. A Next.js frontend built on React provides the user interface. An Express backend exposes REST endpoints and business logic. PostgreSQL stores content, user identities, generated snapshots, and progression data. Prisma acts as the ORM boundary between the backend and the database [4, 5, 6, 7, 8].

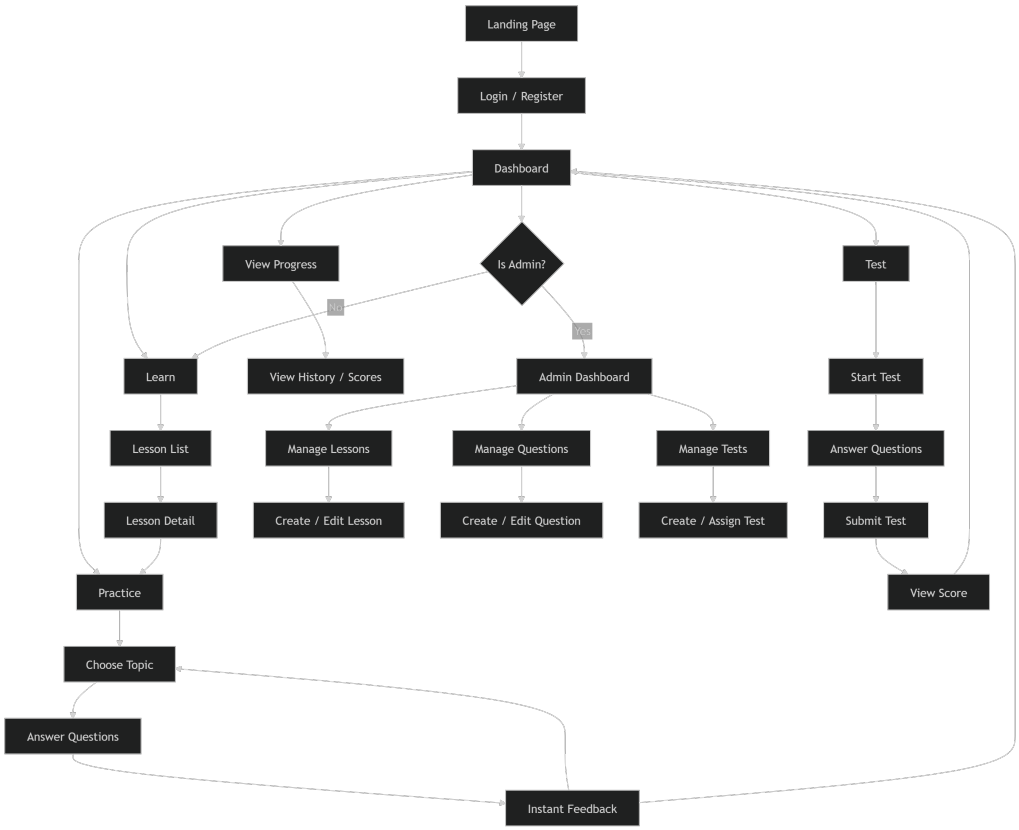

Figure 3.1: User-facing flow of the Anhoc platform

At a high level, the request flow is:

1. the browser interacts with the Next.js frontend;
2. the frontend calls backend routes under `/api/v1`;
3. the backend validates identity and permissions;
4. business services coordinate data access and progression updates;
5. PostgreSQL persists authoritative state.

3.2 Deployment Design

The repository documents a practical deployment split:

- Vercel hosts the frontend runtime;
- Render hosts the Node.js API;
- Neon provides PostgreSQL infrastructure.

This split is appropriate for a full-stack TypeScript project because it separates the static or server-rendered frontend from the API runtime and keeps the relational database independent of either application tier.

3.3 Backend Design

The backend entry point is `backend/server.ts`. It initializes Express, configures CORS, enables JSON parsing, mounts route modules, exposes a health endpoint, and conditionally seeds achievements at startup. The route structure is organized by domain:

- `auth.ts` for identity, login, and profile operations;
- `lessons.ts` for educational content and practice initiation;
- `tests.ts` for attempts, answer submission, template management, and reports;
- `admin.ts` for users, roles, access control, and statistics;
- `achievements.ts` for achievement retrieval and re-evaluation.

This separation keeps HTTP concerns close to route boundaries while pushing calculation-heavy logic into services.

3.4 Frontend Design

The frontend is built with Next.js using an app-router structure under `frontend/src/app`. It includes separate areas for authentication, student flows, and administration. The frontend is supported by Axios-based service modules, Redux state for authentication and permissions, and feature-specific components such as lesson cards, question players, result modals, and achievement galleries. The repository also uses `next-intl` for localized dictionaries and request-scoped internationalization in the App Router [12].

From a design perspective, the frontend acts as an orchestration layer:

- it collects user input and triggers backend operations;
- it renders lesson, practice, test, and admin views;
- it persists authentication tokens in browser storage and cookies;

- it enforces route-level and component-level access decisions using the permission payload returned by the backend.

3.5 Security Model

Authentication is handled with JWT bearer tokens. The middleware in `backend/middleware/auth.ts` extracts the token from the `Authorization` header, verifies it using the configured secret, and attaches the decoded payload to the request object [9, 10].

Authorization is layered on top of authentication. The project uses:

- role identity for coarse-grained decisions;
- grouped resource permissions for action checks;
- subject-level role restrictions for scoped access;
- helper middleware such as `isAdmin` and `selfOrAdmin`.

The design choice to keep permissions data-driven is significant. It allows new resources and actions to be introduced through database records and route logic instead of forcing the codebase into a rigid two-role system.

3.6 Data Model

The database schema in `backend/prisma/schema.prisma` defines the educational, assessment, and progression domain. Table 3.1 summarizes the most important entities.

Table 3.1: Main persistent entities

Entity	Purpose
`users`	Stores account identity and links to roles, attempts, mastery, achievements, and statistics.
`roles`, `actions`, `resources`, `role_permissions`	Implements dynamic role-based access control.
`subjects`, `grades`, `lessons`	Represents the educational content hierarchy.
`question_templates`	Stores reusable question definitions and logic.

Entity	Purpose
test_attempts	Stores headers for practice and test sessions.
question_snapshots	Stores concrete generated questions bound to attempts.
user_lesson_mastery	Stores lesson-level progress and completion status.
student_stats, xp_logs	Stores aggregate progression state and XP history.
achievements, user_achievements	Stores gamification rewards and earned records.
question_template_reports	Stores reports for problematic questions.
audit_logs, activity_evidence	Supports traceability and evidence collection.

3.7 Question Snapshot Strategy

One of the strongest design decisions in Anhoc is the separation between question_templates and question_snapshots. Templates describe how a question can be generated. Snapshots record the exact generated variables, accepted answers, student response, correctness result, and awarded points for one attempt.

This decision solves several engineering problems:

- a generated question does not change after the student starts an attempt;
- grading can be reproduced later without regenerating random variables;
- question reporting can refer to a concrete faulty instance rather than an abstract template;
- analytics can distinguish between template quality and attempt outcomes.

3.8 Progression Design

Progression in Anhoc is not limited to a final score. The backend updates several layers of state:

- lesson mastery based on lesson-linked practice results;
- XP logs for additive progression history;

- aggregate student statistics for dashboards and profile displays;
- achievements that can also grant extra XP.

This design supports both immediate feedback and longitudinal tracking, which is more appropriate for learning software than a simple pass or fail test history.

Chapter 4

Implementation

4.1 Frontend Implementation

The frontend is implemented with Next.js and TypeScript. The app-router structure separates student and administrative workflows into route segments, while service modules in `frontend/src/services` centralize communication with the backend.

Important frontend concerns include:

- authentication bootstrap and persistence;
- protected routes and permission checks;
- lesson and practice page composition;
- test flows with answer submission and result presentation;
- admin screens for users, roles, reports, and dashboards.

The component structure indicates a feature-oriented approach rather than a single monolithic page design. Examples include `QuestionPlayer.tsx`, `PracticeResultModal.tsx`, `AchievementGallery.tsx`, and `CreateTemplateModal.tsx`.

4.2 Backend API Implementation

The backend exposes versioned REST endpoints. Table 4.1 groups the major API areas.

Table 4.1: Main API groups

API Group	Purpose
`/api/v1/auth`	Registration, login, profile retrieval, password change, and activity endpoints.
`/api/v1/lessons`	Lesson, grade, subject, mastery, practice initiation, and study-time endpoints.
`/api/v1/tests`	Grade tests, attempt detail, answer submission, finish flow, template management, and report workflow.
`/api/v1/admin`	Users, roles, access control, insights, and dashboard statistics.
`/api/v1/achievements`	Achievement seeding, listing, earned-state retrieval, and re-checking.

The API design is pragmatic. It is expressive enough for the frontend and explicit enough for debugging, while remaining simple to secure and deploy.

4.3 Question Generation and Grading

The file `backend/services/mathService.ts` contains a central part of the project: logic-driven variable generation and answer evaluation. The implementation uses the `mathjs` expression engine for parsing and evaluating formulas, derived expressions, and constraints [11]. The service supports several rule types:

- fixed numeric values;
- arrays of possible values;
- numeric ranges with optional step and precision;
- random choices from configured lists;
- derived expressions evaluated from previously generated variables;
- constraints that must hold before a question is accepted.

The generation flow can be summarized as:

1. parse the stored `logic_config`;
2. generate variables from configured rules;
3. apply derived expressions;
4. verify constraints;

5. repeat until a valid assignment is found or a maximum number of attempts is reached.

Answer checking supports multiple cases:

- numeric answers, compared with a small tolerance;
- boolean answers, normalized from common textual forms;
- text or structured answers, normalized for comparison;
- multiple accepted formulas for the same question.

This is a stronger implementation than a naive string-equality checker because it tolerates common input variance while still grounding correctness in mathematically evaluated formulas.

4.4 Attempt Lifecycle

The implemented attempt lifecycle is one of the most important interactions in the system.

Practice Start

When a student starts practice for a lesson:

1. the backend validates lesson existence and subject access;
2. it loads lesson templates;
3. it generates variables and right answers for selected templates;
4. it creates a `test_attempt` record;
5. it bulk-inserts `question_snapshots`;
6. it returns a stable attempt payload to the frontend.

Answer Submission

When a student submits an answer:

1. the backend loads the snapshot and template;
2. it evaluates the answer using either text comparison or formula-based checking;
3. it updates the stored snapshot with answer and correctness state;

4. it returns correctness information and explanation content.

Attempt Completion

When an attempt is finished:

1. unanswered snapshots are marked incorrect;
2. correct answers are counted and the final score is computed;
3. lesson mastery is updated when the attempt is linked to a lesson;
4. XP is awarded and logged;
5. aggregate student statistics are recalculated;
6. achievements are re-evaluated and may award additional XP.

This completion pipeline shows that the project treats scoring as part of a broader progression workflow rather than as an isolated calculation.

4.5 Administrative Implementation

The administrative subsystem includes user and role management, subject permission management, dashboard statistics, and question report review. The route design suggests that administrators can change both coarse and fine-grained access behavior without redeploying the application.

This is reinforced by the underlying data model:

- `roles` store named roles;
- `actions` and `resources` define the authorization vocabulary;
- `role_permissions` map actions to resources per role;
- `role_subject_permissions` restrict which subjects a role can manage.

This structure scales better than a hard-coded permission matrix embedded directly in frontend components.

4.6 Configuration and Deployment

The repository defines environment variables for both frontend and backend execution. Examples include:

- API base URLs and rewrite targets for the frontend;
- `DATABASE_URL`, `JWT_SECRET`, and `CORS_ORIGINS` for the backend;
- `AUTO_SEED_ACHIEVEMENTS` to control whether achievement definitions are seeded automatically.

The backend startup process explicitly checks the achievement seeding flag before inserting seed data. This is a sound operational choice because it prevents accidental reseeding in every environment.

Chapter 5

Evaluation and Discussion

5.1 Method of Evaluation

This thesis evaluates Anhoc through implementation analysis and scenario-based validation rather than controlled educational experiments. The repository does not include classroom outcome data, benchmark reports, or a formal automated test suite that would support statistical claims. It would be incorrect to invent such evidence. Instead, the evaluation focuses on whether the implemented design addresses the stated software goals.

5.2 Representative User Scenarios

Scenario 1: Student Login and Study Access

The system supports a full login flow in which credentials are submitted from the frontend, verified by the backend, and returned as a JWT with grouped permissions. This meets the requirement for authenticated access to protected student and admin workflows.

Scenario 2: Lesson-Based Practice

The platform can start practice sessions from lesson templates, generate concrete question instances, and return a stable attempt payload. Because snapshots are persisted, a student can answer the exact generated questions rather than a regenerated variant. This directly satisfies the consistency requirement discussed earlier.

Scenario 3: Assessment and Progression

The finish-attempt flow does more than assign a score. It updates lesson mastery, adds XP, recalculates student statistics, and checks achievements. This makes the system better aligned with long-term learning engagement than a basic quiz engine.

Scenario 4: Administrative Control

The platform exposes dedicated endpoints for roles, users, access control, reports, and dashboard statistics. That breadth indicates the system was designed as a manageable platform rather than a single-user prototype.

5.3 Observed Strengths

Several strengths are visible in the current implementation:

- **Clear layering.** Frontend, backend, services, and database concerns are separated cleanly.
- **Snapshot persistence.** The snapshot model is the strongest architectural decision in the project and improves grading stability, traceability, and debugging.
- **Flexible authorization.** Dynamic RBAC plus subject-level permissions provides useful operational flexibility.
- **Educational progression.** Mastery, XP, levels, and achievements create a richer student model than raw scores alone.
- **Template expressiveness.** Variable ranges, derived expressions, constraints, and accepted formulas make the question system meaningfully reusable.

5.4 Current Limitations

The project also has clear limitations that should be acknowledged directly.

- **No formal experimental validation.** The repository does not provide evidence of student learning impact.
- **Limited compile-time guarantees across dynamic JSON logic.** Template logic is flexible, but JSON-configured expressions can still fail at runtime.

- **Potential duplication of schema ownership.** The repository contains Prisma schema material in both frontend and backend directories, which creates a risk of divergence if not managed carefully.
- **Operational maturity is still emerging.** There is little evidence in the repository of broad automated testing, observability, or release-hardening practices.
- **Reporting and analytics can be expanded.** The current design supports insights and statistics, but a deeper learning analytics layer is not yet visible.

5.5 Risks and Tradeoffs

The design choices in Anhoc involve reasonable tradeoffs:

- Persisting snapshots increases storage cost, but the gain in auditability and reproducibility is worth it.
- Dynamic permissions improve flexibility, but they also require careful synchronization between backend enforcement and frontend affordances.
- Formula-driven generation increases content reusability, but malformed expressions can be harder to validate than fixed question banks.

5.6 Future Improvements

Several next steps would materially improve the project:

- add automated tests for route behavior, template validation, and progression logic;
- build a stronger authoring validation layer for `logic_config` and accepted formulas;
- add richer analytics for question difficulty, error patterns, and lesson effectiveness;
- improve observability through structured logs, metrics, and failure dashboards;
- define a single canonical Prisma schema source and remove duplication risk;
- conduct user studies or classroom pilots to validate educational effectiveness.

5.7 Discussion

Taken as a software engineering project, Anhoc is already beyond a simple tutorial application. It contains meaningful domain modeling, a non-trivial assessment engine, a flexible authorization model, and a progression subsystem tied to learning behavior. Its strongest academic value lies in how it combines educational workflows with practical backend architecture decisions. The platform would benefit from deeper validation and more operational hardening, but the core design is coherent and defensible.

Chapter 6

Conclusion

This thesis documented the design and implementation of Anhoc, a web-based mathematics learning platform that combines bilingual lesson content, generated practice exercises, grade-level assessments, progression tracking, and administrative control. The implemented system uses a modern web stack with a Next.js frontend, an Express backend, Prisma as the persistence layer, and PostgreSQL as the database.

The project demonstrates several sound software engineering decisions. Most importantly, it separates reusable question templates from persisted question snapshots, enabling reproducible grading and better issue reporting. It also uses a dynamic access-control model and a modular progression pipeline for mastery, XP, and achievements. These choices make the system more extensible than a minimal educational CRUD application.

At the same time, the current implementation remains an evolving platform rather than a finished production learning system. It still needs broader automated testing, stronger validation of author-authored template logic, clearer schema ownership, and empirical studies of educational effectiveness. Even with these limitations, the project is a solid foundation for future academic work and practical product development in mathematics learning software.

Appendix A

Appendix

A.1 Build and Run Notes

The repository documents separate frontend and backend setup processes. A representative local workflow is shown below.

```
cd backend
npm install
npm run prisma:migrate
npm run dev
```

```
cd frontend
npm install
npm run dev
```

A.2 Selected Environment Variables

Table A.1: Representative environment variables

Variable	Purpose
NEXT_PUBLIC_API_URL	Browser-visible API base URL for the frontend.
INTERNAL_API_URL	Server-side fetch base used by the frontend runtime.
BACKEND_URL	Optional rewrite target for same-origin frontend calls.
SERVER_PORT	Backend listening port.
DATABASE_URL	PostgreSQL connection string for Prisma.
JWT_SECRET	Secret used to verify and sign JWT tokens.
CORS_ORIGINS	Allowed origins for cross-origin backend requests.
AUTO_SEED_ACHIEVEMENTS	Controls automatic achievement seeding on startup.

A.3 Selected REST Endpoints

Table A.2: Example endpoints from the implemented API

Method	Path	Purpose
POST	/api/v1/auth/login	Authenticate a user and return token plus permission payload.
POST	/api/v1/lessons/:id/practice	Start a lesson-based practice attempt.
POST	/api/v1/tests/submit-answer	Submit an answer for a generated question snapshot.
POST	/api/v1/tests/attempts/:id/finish	Complete an attempt and trigger progression updates.
GET	/api/v1/admin/access-control	Retrieve data needed for role and permission administration.

Bibliography

[1] Anhoc project repository overview, `README.md`.

[2] Anhoc frontend package manifest and framework dependencies, `frontend/package.json`.

[3] Anhoc backend package manifest and runtime dependencies, `backend/package.json`.

[4] Vercel. *Next.js Documentation: App Router*. Available at: https://nextjs.org/docs/app. Accessed June 15, 2026.

[5] Meta Platforms, Inc. and contributors. *React Documentation*. Available at: https://react.dev/learn. Accessed June 15, 2026.

[6] OpenJS Foundation and Express contributors. *Express.js 5.x API Reference*. Available at: https://expressjs.com/en/api/. Accessed June 15, 2026.

[7] Prisma. *Prisma ORM Documentation*. Available at: https://www.prisma.io/docs/orm. Accessed June 15, 2026.

[8] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*. Available at: https://www.postgresql.org/docs/current/index.html. Accessed June 15, 2026.

[9] M. Jones, J. Bradley, and N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. RFC Editor, May 2015. Available at: https://www.rfc-editor.org/info/rfc7519/.

[10] N. Sheffer, D. Hardt, and M. Jones. *RFC 8725: JSON Web Token Best Current Practices*. RFC Editor, February 2020. Available at: https://www.rfc-editor.org/info/rfc8725/.

[11] *mathjs Documentation*. Available at: https://mathjs.org/docs/. Accessed June 15, 2026.

[12] Jan Amann and contributors. *next-intl Documentation: Getting Started*. Available at: https://next-intl.dev/docs/getting-started. Accessed June 15, 2026.

[13] Anhoc backend bootstrap implementation, `backend/server.ts`.

[14] Anhoc database schema and Prisma models, `backend/prisma/schema.prisma`.

[15] Anhoc authentication and authorization middleware, `backend/middleware/auth.ts`.

[16] Anhoc math generation and answer evaluation service, `backend/services/mathService.ts`.

[17] Anhoc tutoring chatbot specification and implementation, `chatbot/SPEC.md`, `chatbot/README.md`, and `chatbot/main.py`.

[18] Anhoc tutoring widget integration in the frontend, `frontend/src/components/feature/ChatbotWidget.tsx`.

[19] Anhoc technical design document, `docs/TDD/technical-design.md`.